Supplementary Material

Adaptation-Aware Model Selection for Few-Shot Digital Twin Instantiation

S1 Task chronology and measurement

Each Bitbrains VM belongs to one of six groups. Twenty source VMs supply training windows. Four separate groups of 20 provide the screening reference (A), screening evidence (B), deployment reference (C), and replacement margin (D). The remaining 30 VMs provide the evaluation tasks (E). Each VM contributes one task for each combination of forecast horizon $H \in \{1, 4\}$ and support size $K \in \{10, 20\}$. The file `data/roles.json` identifies each VM. The comparison is retrospective. The evaluation group was also examined in earlier experiments.

An input contains the 96 records ending at its forecast origin. The target is CPU utilization over the next H records. Only the K support windows supply labels for parameter updates. The validation history used to compare candidates ends at the decision time T. It provides 1,342–1,351 windows before masking (Table S1). Once a model is selected, its parameters stay fixed as new observations enter subsequent forecast inputs.

Table S1: Task windows in zero-based record indices.

H	K	Validation origins	Windows before masking	T	Forecast origins
1	10	201–1551	1351	1552	1552–2178
1	20	211–1556	1346	1557	1557–2181
4	10	204–1549	1346	1553	1553–2176
4	20	214–1555	1342	1559	1559–2179

The four CPU channels record core count, provisioned capacity, usage in MHz, and utilization in percent. Six others describe provisioned and used memory, disk read and write throughput, and incoming and outgoing network throughput. Each channel has a validity indicator. Finite, nonnegative values are valid. Missing values carry forward the last valid observation, starting from zero.

CPU utilization and its labels are divided by 100. For each of the other nine raw channels, we divide $\log(1 + x)$ by $\max(q_{.99}, 10^{-6})$. Here $q_{.99}$ is the 99th percentile of valid log-transformed observations in the first 24 hours of the 20 source VMs. Two derived features track the trailing five-record mean of CPU utilization and its change since the preceding record. Adding their validity indicators and the time gap in units of 1,200 seconds gives 25 inputs in total. On Azure, the indicators identify the nine unavailable raw channels.

Each derived feature is valid only when all raw CPU values it uses are valid. The initial difference and time gap are zero. Recorded CPU utilization above 100% remains valid. The transformations use only current and past records.

Only forecast windows with all H CPU targets observed enter the error calculation. Each window contributes its mean squared or absolute error across the forecast horizon. Task MSE and MAE average over that task's valid windows. Worst-10% averages its largest 10% of window-level squared errors, rounded to the nearest whole count with at least one window. Reported means weight VMs equally after averaging the four tasks within each VM. CVaR90 instead averages the 12 largest task MSEs among the 120 tasks.

S2 Evidence for retaining an architecture

Each source VM supplies 100 training windows per horizon from its first 24 hours. We trained 66 architectures separately for each horizon. In each of 50 epochs, Adam visits each source VM once and processes its windows in batches of 64 and 36. It minimizes MSE at a learning rate of 0.001. On a target task, SGD uses all support windows at each update, with learning rate 0.01 and gradient-norm limit 1.

A55 had the lowest mean validation MSE after adaptation in group A. It met the limits at every budget. In group B, we compared each feasible challenger with A55 on the same task. The challenger with the lowest error on subsequent check windows earns a check point if it beats A55 there. The challenger with the lowest validation error earns a validation point if it beats A55 on both validation and check windows. The bank in Table S2 includes A55 and any challenger that earned at least two points of the same type. Each retained architecture keeps its original source weights.

Table S2: Retained source checkpoints and group-B evidence.

ID	Architecture	Check wins	Val. wins	Parameters $H = 1/4$	Est. operations (10^6) per forecast ($H = 1/4$)
A0	MLP_L2_H32_D0.0	8	6	77,921 / 78,020	0.156 / 0.156
A6	MLP_L3_H32_D0.0	8	10	78,977 / 79,076	0.158 / 0.158
A8	MLP_L3_H64_D0.0	2	2	162,049 / 162,244	0.324 / 0.324
A13	MLP_L4_H32_D0.1	2	0	80,033 / 80,132	0.160 / 0.160
A27	TCN_B2_C32_K3_Dil2	2	0	12,609 / 12,708	2.384 / 2.384
A31	TCN_B3_C8_K3_Dil2	0	2	1,825 / 1,852	0.338 / 0.338
A32	TCN_B3_C8_K5_Dil1	17	15	2,865 / 2,892	0.538 / 0.538
A35	TCN_B3_C16_K3_Dil2	8	8	5,569 / 5,620	1.045 / 1.045
A40	TCN_B3_C32_K5_Dil1	6	6	30,657 / 30,756	5.837 / 5.837
A45	TCN_B4_C8_K5_Dil2	6	1	3,521 / 3,548	0.660 / 0.661
A46	TCN_B4_C16_K3_Dil1	2	3	7,137 / 7,188	1.339 / 1.340
A47	TCN_B4_C16_K3_Dil2	2	4	7,137 / 7,188	1.339 / 1.340
A50	TCN_B4_C32_K3_Dil1	0	2	25,025 / 25,124	4.743 / 4.743
A54	GRU_L1_H16_D0.0	2	4	2,081 / 2,132	0.378 / 0.378
A55	GRU_L1_H16_D0.1	0	0	2,081 / 2,132	0.378 / 0.378
A57	GRU_L1_H32_D0.1	0	2	5,697 / 5,796	1.051 / 1.051
A58	GRU_L1_H64_D0.0	2	0	17,537 / 17,732	3.281 / 3.281
A61	GRU_L2_H16_D0.1	1	2	3,713 / 3,764	0.673 / 0.673
A63	GRU_L2_H32_D0.1	2	2	12,033 / 12,132	2.230 / 2.231

Inter-layer dropout is inactive in a one-layer GRU. A54 and A55 therefore share an executable structure but retain different source weights.

As a check on how much evidence a smaller screening group supplies, we sorted the 20 group-B VMs by identifier and split them into groups of seven, seven, and six. Applying the same two-point requirement to these groups gave 10, 10, and 7 retained models, respectively, including A55. The full group retained 19. These counts describe screening only.

S3 Choosing and replacing the deployment reference

Group C compares the retained candidates that meet both resource limits at every budget and horizon. A57 had the lowest mean post-adaptation assessment MSE among these ten candidates (0.003420). Group D then measures what happens when an alternative replaces the adapted A57. For each candidate margin, the choice uses validation predictions. A replacement is harmful if its later assessment MSE exceeds that of the adapted reference. The harm rates in Table S3 use all 80 tasks as the denominator.

Table S3: Group-D calibration on 80 tasks at the medium budget.

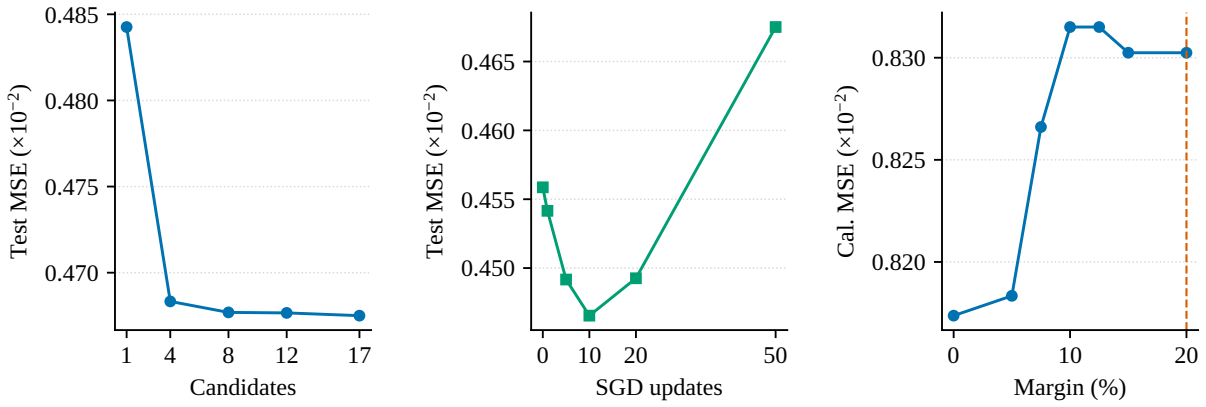
Margin (%)	Assessment MSE (10^{-2})	Harmful replacements	Replacements
0	0.8174	8/80	80/80
5	0.8183	6/80	74/80
7.5	0.8266	6/80	69/80
10	0.8315	6/80	68/80
12.5	0.8315	6/80	68/80
15	0.8302	3/80	65/80
20	0.8302	3/80	65/80

Among the margins with a harm rate at or below 5%, 15% and 20% gave the same lowest assessment MSE. The prespecified tie rule favors the larger margin. The resulting 20% margin applies at all three budgets. At the medium budget in group E, 83 replacements improved on A57, two raised test MSE, and 35 tasks kept A57. The 5% calibration criterion refers to the observed group-D tasks.

At the medium budget, the ratio of the two methods' mean errors gives a 6.04% MSE reduction relative to fixed PT-FT. The paired 95% bootstrap interval is 2.00–22.38%. We resampled VMs 10,000 times, keeping all four tasks of each sampled VM together. Calculating the reduction within each task before averaging gives 54.83%. This calculation weights each percentage equally even when PT-FT errors differ greatly.

S4 Candidate subsets and target updates

The comparisons in Figure S1 and Table S4 use the same source checkpoints and 120 medium-budget test tasks. Every subset includes A57. To add candidates, we rank them by the larger of their two group-B counts. Ties favor more check wins, then more validation wins, then the lower architecture index. Each subset and each update count has its own margin, calibrated on the 80 group-D tasks.



(a) Candidate subsets (b) Target updates (c) Calibration

Figure S1: Mean evaluation MSE as the candidate set or number of updates changes. The last panel reports calibration MSE. The dashed line marks the selected margin.

With only A57 available, mean test MSE was 0.004843. With the full set of 17 feasible candidates it was 0.004675. With the same 17 candidates but no target updates, MSE was 0.004559.

We adapted all 17 models for 50 updates with either SGD (learning rate 0.01) or Adam (0.001). For each optimizer, direct selection chose the lowest validation MSE. The calibrated rule applied the replacement margin to the same validation scores.

Table S4: Optimizer-matched selection at the medium budget.

Update and selection rule	Margin (%)	MSE (10^{-2})	MAE (10^{-2})	Harm / 120
SGD, calibrated	20	0.4675	1.9509	2
SGD, direct minimum	0	0.4519	1.9305	13
Adam, calibrated	7.5	0.4792	1.9953	2
Adam, direct minimum	0	0.4782	2.0051	5

S5 Sensitivity to source training

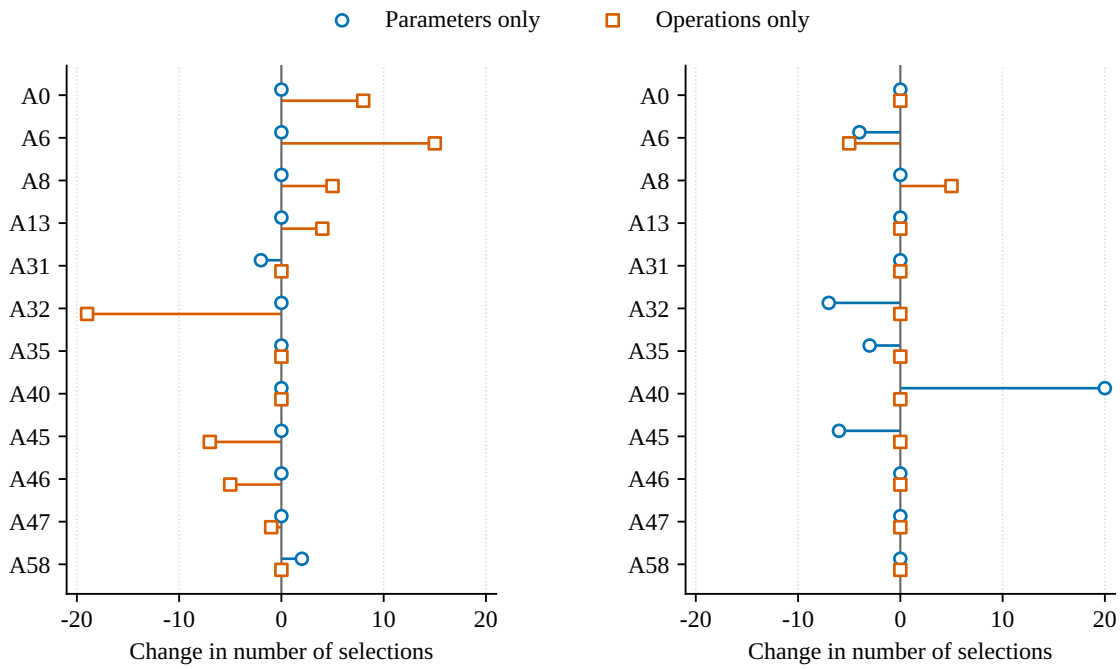
The source pools in Table S5 use nested prefixes of the same 20 source VMs. All conditions share the original feature scales. Smaller pools retain the 50-epoch recipe, so fewer VMs also mean fewer training windows and optimizer updates. Two further runs keep all 20 VMs and change the source seed. For each condition, we repeated screening, selection of the deployment reference and calibration on the original A–D groups. We counted retained checkpoints before budget filtering and evaluated harmful replacements against that run's adapted deployment reference. Group E is used only after these choices have been fixed.

Table S5: Effect of source-pool size and seed on medium-budget evaluation.

VMs	Seed	Screen ref.	Bank	Deploy ref.	Margin (%)	MSE (10^{-2})	MAE (10^{-2})	Harm / 120
5	2904	A56	27	A56	7.5	1.3350	3.7131	1
10	2904	A47	24	A22	Reference only	0.6221	3.0418	0
15	2904	A57	21	A57	5	0.4494	2.0358	2
20	2904	A55	19	A57	20	0.4675	1.9509	2
20	2905	A54	20	A56	Reference only	0.4843	2.3090	0
20	2906	A30	18	A30	15	0.4598	1.8816	2

S6 Which resource limits affect the final choice?

We applied the parameter limit alone, the operation limit alone, or both to the same 19 candidates. All three conditions used A57 and the 20% margin. We checked each selected model against both limits, including the limit omitted during selection. Figure S2 compares selection frequencies. Table S6 reports compliance and error for the same paired tasks.



(a) Tight budget

(b) Medium budget

Figure S2: Change in selection frequency when only one limit is enforced. Positive values indicate more selections than when both limits apply. The loose budget admits the full bank.

At the tight budget, omitting the parameter limit produced 32 outputs above that limit. Omitting the operation limit gave 2 outputs above the operation limit. Test MSE in the table includes these outputs.

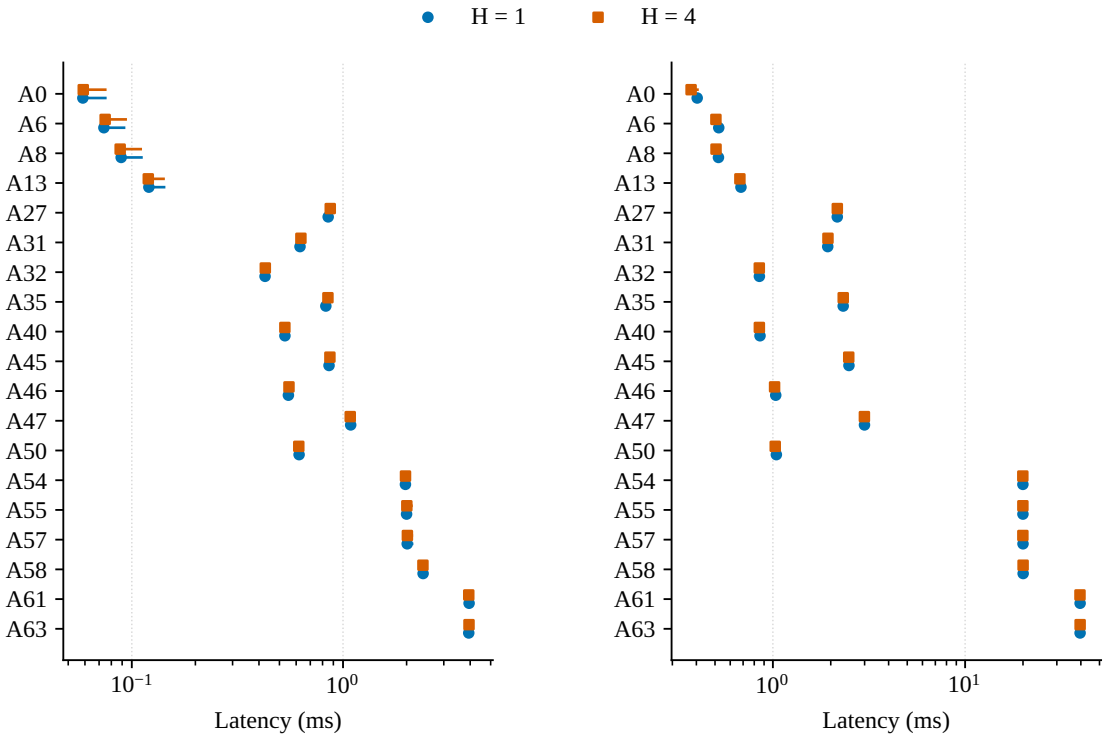
Table S6: Resource compliance and test error under each filtering rule.

Budget	Active limits	Candidates	Meet both limits	MSE (10^{-2})	Harm / 120
Tight	Parameters	14	118/120	0.4682	0
Tight	Operations	14	88/120	0.4677	2
Tight	Both	10	120/120	0.4684	0
Medium	Parameters	18	100/120	0.4679	4
Medium	Operations	18	115/120	0.4675	2
Medium	Both	17	120/120	0.4675	2
Loose	Parameters	19	120/120	0.4679	4
Loose	Operations	19	120/120	0.4679	4
Loose	Both	19	120/120	0.4679	4

The parameter count includes the horizon-specific output head. The operation estimate covers dense layers, convolutions, recurrent layers, and residual projections. A multiply and an addition count as two operations. Bias additions and elementwise activations are excluded.

S7 Inference latency and target selection time

Latency in Figure S3 was measured on an Intel Xeon Processor (Skylake, IBRS) CPU with one thread and a Tesla V100S-PCIE-32GB GPU using PyTorch 2.5.1+cu121. A single zero-valued input has shape $1 \times 96 \times 25$. Each checkpoint receives 100 warmup calls followed by five blocks of 1,000 forward passes. We synchronize the GPU before and after each timed call. The input and model are already on the measurement device. GRUs use the native backend used during adaptation.



(a) CPU, one thread

(b) GPU, synchronized

Figure S3: Single-window inference latency. Symbols mark medians and lines extend to the 95th percentile. The horizontal axes show milliseconds on logarithmic scales.

The timings in Table S7 use VM 13, the lowest numbered VM in group D. For each task and budget, we ran one warmup and five timed selections on an otherwise idle GPU. Timing starts with resource filtering and checkpoint loading, with task features already constructed. It includes adaptation, validation, and the final choice. Fixed PT-FT adapts A39 at every budget. At the tight budget its time describes execution of a model that exceeds the operation limit.

Table S7: Mean target selection time in seconds over five repetitions.

		Tight		Medium		Loose	
H	K	MSA-DTI	PT-FT	MSA-DTI	PT-FT	MSA-DTI	PT-FT
1	10	30.234	4.424	44.053	4.438	44.344	4.430
1	20	35.503	5.971	50.477	5.994	50.902	5.971
4	10	30.217	4.428	44.159	4.451	44.368	4.427
4	20	35.537	5.966	50.546	5.957	50.940	5.966

Table 4 in the main article also includes source training and development.

The source package includes the task groups, budget limits, architecture profiles, and plotting scripts. The directory `data/supplement/` contains the task selections and timing summaries, including the standard deviation and range for each condition. The experiment archives retain the new source checkpoints, selection records, and predictions.